

A Feasibility Study for Building a Whole-Cell Model of the Human Platelet

Using the wcEcoli Framework as a Foundation

Steve Haigh

April 2026

This document synthesises the rationale, technical analysis, and implementation plan for adapting the wcEcoli whole-cell modelling framework to simulate human platelet biology.

Contents

I	Background: Whole-Cell Modelling Approaches	4
1	Introduction to Whole-Cell Modelling	5
1.1	What is Whole-Cell Modelling?	5
1.2	Why Model Platelets?	5
2	wcEcoli: The E. coli Whole-Cell Model	6
2.1	Overview	6
2.1.1	Key References	6
2.1.2	What It Simulates	6
2.2	Inputs: Data and Parameter Fitting	6
2.2.1	Raw Data (72+ TSV Files)	6
2.2.2	Parameter Calculator (ParCa)	6
2.3	The Simulation Loop	7
2.3.1	Timestep Structure	7
2.3.2	Process Groups and Ordering	7
2.3.3	Resource Partitioning	8
2.4	Major Processes	8
2.4.1	Transcription	8
2.4.2	Translation	8
2.4.3	Metabolism	8
2.4.4	DNA Replication	8
2.5	Outputs	9
2.5.1	Directory Structure	9
2.5.2	Listener Data (18 Listeners)	9
3	MC4D: The 4D Minimal Cell Model	10
3.1	Overview	10
3.1.1	The Organism	10
3.2	The Four Computational Methods	10
3.2.1	RDME — Reaction-Diffusion Master Equation	10
3.2.2	CME — Chemical Master Equation	10
3.2.3	ODE — Ordinary Differential Equations	10
3.2.4	BD — Brownian Dynamics	10
3.3	Integration Architecture	11
3.4	Validation	11
3.5	Technical Requirements	11
4	Comparison: wcEcoli vs MC4D	12
4.1	Side-by-Side Overview	12
4.2	Key Architectural Differences	12
4.2.1	Spatial Resolution	12
4.2.2	Metabolic Treatment	12
4.2.3	Stochasticity	12
4.2.4	Framework Design	12

4.3	Implications for Platelet Modelling	12
II	Technical Analysis: Framework Reusability	14
5	wcEcoli Architecture for Reuse	15
5.1	High-Level Structure	15
5.2	Simulation Engine (Fully Generic)	15
5.3	State Classes	15
5.3.1	BulkMolecules	15
5.3.2	UniqueMolecules	15
5.3.3	LocalEnvironment	16
5.4	Process Architecture	16
5.5	Reusability Summary	16
5.6	Minimum sim_data Interface	16
III	The Platelet Model	18
6	Biological Context	19
6.1	Key Differences from E. coli	19
6.2	Platelet-Specific Biology to Model	19
6.2.1	Compartmentalisation	19
6.2.2	Receptors and Signalling	19
6.2.3	Calcium Dynamics	20
6.2.4	Granule Exocytosis	20
6.2.5	Metabolism	20
7	Literature Analysis	21
7.1	Key Data Sources	21
7.1.1	Calcium Dynamics	21
7.1.2	Receptor Signalling	21
7.1.3	Granule Exocytosis	21
7.1.4	Metabolism	21
7.1.5	Proteomics	21
8	Implementation Plan	22
8.1	Phased Approach	22
8.1.1	Phase 1: Framework Validation (2–3 weeks)	22
8.1.2	Phase 2: Minimal Platelet Stub (3–4 weeks)	22
8.1.3	Phase 3: Biological Processes (8–12 weeks)	22
8.1.4	Phase 4: Integration and Validation (4+ weeks)	23
8.2	v0.1 Target Definition	23
8.3	Proposed File Structure	24
9	Feasibility Assessment	25
9.1	Technical Feasibility	25
9.1.1	Framework Suitability	25
9.1.2	Biological Compatibility	25
9.1.3	Risk Assessment	25
9.2	Scientific Feasibility	25
9.2.1	Data Availability	25
9.2.2	Model Scope	26

9.3	Resource Requirements	26
9.3.1	Computational	26
9.3.2	Time Estimate	26
9.4	Conclusion	26
IV	Appendices	28
A	Key Literature References	29
A.1	Whole-Cell Modelling	29
A.2	Platelet Calcium Signalling	29
A.3	Platelet Receptor Signalling	29
A.4	Platelet Metabolism	29
A.5	Platelet Proteomics	29
B	Glossary	30

Part I

Background: Whole-Cell Modelling Approaches

Chapter 1

Introduction to Whole-Cell Modelling

1.1 What is Whole-Cell Modelling?

Whole-cell modelling represents a paradigm shift in computational biology. Rather than modelling individual pathways or processes in isolation, a whole-cell model (WCM) integrates *all* major cellular processes into a single, self-consistent simulation. The goal is to predict cellular phenotype directly from molecular-level mechanisms—to simulate the entire life cycle of a cell from first principles.

The key characteristics of whole-cell models are:

- **Comprehensive scope:** All major biological processes (metabolism, gene expression, replication, etc.) are included.
- **Mechanistic detail:** Processes are modelled with sufficient biochemical detail to capture the underlying biology.
- **Integration:** Processes share a common molecular inventory and influence each other dynamically.
- **Emergent behaviour:** System-level properties (growth rate, cell cycle timing) emerge from the simulation rather than being imposed.

This document examines two state-of-the-art whole-cell models—wcEcoli and MC4D (the 4D minimal cell model)—and presents a feasibility study for adapting the wcEcoli framework to model human platelets.

1.2 Why Model Platelets?

Platelets are clinically critical cells involved in haemostasis, thrombosis, and inflammation. Despite extensive biochemical characterisation, no integrative whole-cell model of the platelet exists. Current computational models focus on individual pathways:

- Calcium signalling models (Sveshnikova, Diamond groups)
- Coagulation cascade models (cascade kinetics)
- Integrin activation models (inside-out signalling)

A whole-cell model would enable questions that span pathways:

1. How does receptor activation propagate through signalling cascades to granule release?
2. How does platelet energetics (ATP supply) constrain activation kinetics?
3. What explains heterogeneity in activation thresholds across individual platelets?

These are precisely the cross-pathway, multi-scale questions that whole-cell modelling is designed to address.

Chapter 2

wcEcoli: The *E. coli* Whole-Cell Model

2.1 Overview

wcEcoli is an open-source whole-cell computational model of *Escherichia coli*, developed by the Covert Lab at Stanford University. It represents the most complete whole-cell model of a free-living organism currently available.

2.1.1 Key References

- Karr JR et al. (2012) “A whole-cell computational model predicts phenotype from genotype.” *Cell* 150(2):389–401.
- Macklin DN et al. (2020) “Simultaneous cross-evaluation of heterogeneous *E. coli* datasets via mechanistic simulation.” *Science* 369:eaav3751.
- Ahn-Horst TA et al. (2022) “An expanded whole-cell model of *E. coli* links cellular physiology with mechanisms of growth rate control.” *npj Syst Biol Appl* 8:30.

2.1.2 What It Simulates

The model simulates a single *E. coli* cell from birth to division (60–120 minutes), tracking:

- **Every gene** (~4,600 genes) and transcription state
- **Every mRNA molecule** (~2,000–8,000 copies) and degradation
- **Every protein** (~2.4 million molecules) synthesis and turnover
- **Every metabolite** (~1,500 species) concentrations and fluxes
- **DNA replication** fork positions and chromosome copy number
- **Cell growth** from ~650 fg to ~1,300 fg dry mass at division

2.2 Inputs: Data and Parameter Fitting

2.2.1 Raw Data (72+ TSV Files)

The model is parameterised from curated experimental data stored in `reconstruction/ecoli/flat/`:

2.2.2 Parameter Calculator (ParCa)

Raw data is transformed by `fit_sim_data_1.py` through a multi-stage fitting pipeline:

1. `initialize()` — Load and validate raw data
2. `basal_specs()` — Calculate basal expression levels
3. `tf_condition_specs()` — Fit TF regulation parameters

Category	Example Files	Contents
Genome	genes.tsv, sequence.fasta	4,583 genes, coordinates
Transcription	rnas.tsv, transcription_factors.tsv	RNA properties, TF binding
Translation	proteins.tsv, translation_efficiency.tsv	Protein sequences, rates
Metabolism	metabolic_reactions.tsv, metabolism_kinetics.tsv	~2,500 reactions
Regulation	fold_changes.tsv, ppgpp_regulation.tsv	TF effects, stress response

Table 2.1: Categories of raw data used to parameterise wcEcoli.

4. `fit_condition()` — CVXPY optimisation of expression
5. `promoter_binding()` — Calculate TF–promoter binding probabilities
6. `final_adjustments()` — Calibrate to target growth rate

The output is a ~180 MB serialised `sim_data` object containing all fitted parameters.

2.3 The Simulation Loop

2.3.1 Timestep Structure

Each simulation timestep (default 1 second, dynamically adjusted 0.1–1.0s):

1. **Calculate Requests:** Each process declares what molecules it needs.
2. **Partition State:** Molecules allocated to processes by priority.
3. **Evolve State:** Processes execute their biological function.
4. **Merge State:** Updates combined back into global state.
5. **Update Listeners:** Record data to disk for analysis.

2.3.2 Process Groups and Ordering

Processes run in dependency-ordered groups:

```

_processClasses = (
    (TfUnbinding,),                                # Group 1: Must run first
    (Equilibrium, TwoComponentSystem, RnaMaturation), # Group 2
    (TfBinding,),                                  # Group 3: Before transcription
    (TranscriptInitiation, PolypeptideInitiation,
     ChromosomeReplication, ProteinDegradation,
     RnaDegradation, Complexation),                # Group 4: Initiation + decay
    (TranscriptElongation, PolypeptideElongation),  # Group 5
    (ChromosomeStructure,),                          # Group 6
    (Metabolism,),                                  # Group 7: Balances all changes
    (CellDivision,),                                # Group 8: Checks completion
)

```


2.3.3 Resource Partitioning

Processes compete for molecules through priority-based allocation:

- REQUEST_PRIORITY_METABOLISM = -10 (highest priority)
- REQUEST_PRIORITY_DEFAULT = 0
- REQUEST_PRIORITY_DEGRADATION = 10 (lowest priority)

When requests exceed availability, proportional allocation with stochastic rounding is used.

2.4 Major Processes

2.4.1 Transcription

TranscriptInitiation: Calculates synthesis probability for each gene as:

$$P(\text{synthesis}) = P_{\text{basal}} \times \text{TF modulation} \times \text{ppGpp effect}$$

A multinomial draw determines which genes are initiated each timestep.

TranscriptElongation: Advances RNAP positions along DNA, consuming NTPs. When RNAP reaches gene end, a new RNA molecule is created.

2.4.2 Translation

Uses tRNA charging dynamics to model ribosome elongation:

1. Calculate amino acid demand from sequences being translated
2. Solve tRNA charging kinetics (steady-state approximation)
3. Elongation rate depends on charged tRNA availability
4. Stochastic polymerisation with resource constraints

Key biological behaviours captured include ribosome stalling, codon-dependent rates, and competition between mRNAs.

2.4.3 Metabolism

Uses Flux Balance Analysis (FBA):

1. Build stoichiometric matrix S (metabolites $\times$ reactions)
2. Set flux bounds from enzyme counts and nutrient availability
3. Objective: maximise biomass production
4. Solve linear program to find optimal flux distribution

The steady-state assumption ($S \cdot v = 0$) is implicit in FBA.

2.4.4 DNA Replication

Tracks replication fork positions, with rates determined by dNTP availability. Initiation triggered by DnaA-ATP concentration threshold.

2.5 Outputs

2.5.1 Directory Structure

```

out/{sim_dir}/
  kb/simData.cPickle          # Fitted parameters (180 MB)
  wildtype_000000/           # Variant
    000000/                   # Seed
      generation_000000/
        000000/               # Daughter
          simOut/              # Listener data
            plotOut/           # Analysis figures

```

2.5.2 Listener Data (18 Listeners)

Each listener records specific data each timestep:

Listener	Data Recorded
Mass	Total mass, dry mass, mass fractions
BulkMolecules	Counts of all ~3,000 molecule species
FBAResults	Reaction fluxes, exchange rates
GrowthLimits	Which resources limit growth
RibosomeData	Active ribosomes, stalling events
ReplicationData	Fork positions, chromosome count

Table 2.2: Key listeners in wcEcoli.

Chapter 3

MC4D: The 4D Minimal Cell Model

3.1 Overview

MC4D is a whole-cell model of JCVI-syn3A, a synthetically constructed minimal bacterium with approximately 473 genes. Published in *Cell* (March 2026), it is the first whole-cell model to integrate full 3D spatial resolution with a complete biological description.

3.1.1 The Organism

JCVI-syn3A was engineered by iteratively removing genes from *Mycoplasma mycoides*, retaining only those essential for growth and division. With ~ 473 genes, it is dramatically simpler than *E. coli* ($\sim 4,300$ genes), making complete gene coverage tractable. However, roughly one third of genes have unknown function.

3.2 The Four Computational Methods

The model is named “4D” both for its spatial dimensions and for the four mathematical formalisms it integrates:

3.2.1 RDME — Reaction-Diffusion Master Equation

The cell interior is discretised onto a 3D lattice. Molecules diffuse stochastically between voxels and react based on local concentrations. Implemented via Lattice Microbes (GPU-accelerated).

Used for: Translation (spatially resolved), molecular diffusion, ribosome dynamics with excluded-volume constraints.

3.2.2 CME — Chemical Master Equation

A well-mixed stochastic formalism for globally-treated reactions where spatial homogeneity is reasonable but discrete-molecule stochasticity matters.

Used for: Transcription, tRNA charging.

3.2.3 ODE — Ordinary Differential Equations

Deterministic kinetic integration of the metabolic network. Enzyme concentrations from RDME are passed to the ODE solver.

Used for: Metabolism, dNTP pool dynamics.

3.2.4 BD — Brownian Dynamics

Explicit physical simulation of chromosomal DNA as a polymer, using LAMMPS molecular dynamics. SMC proteins and topoisomerases control compaction and segregation.

Used for: Chromosome structure and segregation.

3.3 Integration Architecture

The four methods communicate through a central Python module (`Hook.py`) that periodically:

1. Extracts molecular counts from the RDME lattice
2. Passes enzyme concentrations to ODE solver; runs metabolism
3. Passes transcription events to CME solver
4. Synchronises chromosome state with BD simulation
5. Writes updated counts back to RDME lattice

3.4 Validation

The model is validated against:

- Doubling time (quantitative agreement)
- Origin-to-terminus ratio (DNA sequencing data)
- mRNA half-lives (experimental measurements)
- Protein spatial distributions (fluorescence data)
- Ribosome counts (quantitative agreement)
- Daughter cell heterogeneity (stochastic partitioning)

3.5 Technical Requirements

Component	Software	Notes
RDME solver	Lattice Microbes	GPU required
ODE integrator	odecell	Python-based
Chromosome dynamics	btree_chromo + LAMMPS	Kokkos-enabled build
Orchestration	Hook.py	Central integration

Table 3.1: Software dependencies for MC4D.

Chapter 4

Comparison: wcEcoli vs MC4D

4.1 Side-by-Side Overview

Dimension	wcEcoli	MC4D
Organism	<i>E. coli</i> (~4,300 genes)	JCVI-syn3A (~473 genes)
Gene coverage	~43% of characterised genes	~100% (including unknowns)
Spatial resolution	None (well-mixed)	Full 3D (RDME lattice + BD)
Cell cycle	25–130 min	~100 min
Metabolism	Flux Balance Analysis	ODE kinetic network
Transcription	Stochastic, well-mixed	CME, global stochastic
Translation	Stochastic, well-mixed	RDME, spatially resolved
Chromosome	Position tracking only	Brownian dynamics polymer
GPU required	No	Yes
Framework reuse	Explicitly designed for it	Research prototype

Table 4.1: Comparison of wcEcoli and MC4D.

4.2 Key Architectural Differences

4.2.1 Spatial Resolution

wcEcoli treats the cell as well-mixed compartments (cytoplasm, periplasm, extracellular). MC4D places every molecule on a 3D lattice with explicit diffusion. This reveals spatial heterogeneity but requires GPU acceleration.

4.2.2 Metabolic Treatment

wcEcoli uses FBA (steady-state assumption, no kinetic parameters required). MC4D uses ODE kinetics (captures transients, requires rate constants for every reaction).

4.2.3 Stochasticity

Both models are stochastic, but MC4D more thoroughly so. All molecular processes in MC4D are governed by stochastic formalisms, producing natural population heterogeneity.

4.2.4 Framework Design

wcEcoli explicitly separates the generic engine (`wholecell/`) from organism-specific biology (`models/ecoli/`). MC4D is a research prototype not architected for reuse.

4.3 Implications for Platelet Modelling

Pragmatic conclusion: Build the initial platelet model using wcEcoli’s framework (fast, reusable, well-tooled), using ODE kinetics for calcium/signalling rather than FBA. If spatial

Consideration	wcEcoli	MC4D
Spatial heterogeneity	Limited (well-mixed)	Excellent (RDME)
Transient dynamics	Limited (FBA steady-state)	Excellent (ODE kinetics)
Chromosome modelling	Not needed for platelets	Not needed for platelets
Framework reusability	High	Low
Accessibility	Runs on laptop	Requires GPU + LAMMPS

Table 4.2: Relevance of each model to platelet modelling.

effects prove critical, the RDME approach from MC4D could inform a second-generation model.

Part II

Technical Analysis: Framework Reusability

Chapter 5

wcEcoli Architecture for Reuse

5.1 High-Level Structure

The wcEcoli framework has four layers:

1. **Reconstruction** — builds `sim_data` from raw data
2. **Simulation engine** — generic time-stepping loop
3. **Organism model** — processes, states, listeners (E. coli-specific)
4. **Analysis** — post-simulation plotting

5.2 Simulation Engine (Fully Generic)

The base `Simulation` class (`wholecell/sim/simulation.py`) requires subclasses to define:

```
class Simulation:
    _definedBySubclass = (
        "_internalStateClasses",      # State containers
        "_externalStateClasses",      # Environment
        "_processClasses",            # Biological processes
        "_initialConditionsFunction",  # Set starting state
    )
    # Optional:
    _listenerClasses = ()
    _divideCellFunction = None  # Set to None for non-dividing cells
```

Reusability: FULLY GENERIC. No organism-specific logic.

5.3 State Classes

5.3.1 BulkMolecules

Tracks integer copy numbers in a flat array. Interface with `sim_data`:

```
sim_data.internal_state.bulk_molecules.bulk_data['id']      # molecule IDs
sim_data.internal_state.bulk_molecules.bulk_data['mass']    # masses
sim_data.molecule_groups.bulk_molecules_binomial_division # division
lists
```

Reusability: HIGH. One hardcoded reference to `ATP[c]` needs generalisation.

5.3.2 UniqueMolecules

Tracks individual molecules with per-instance attributes (structured numpy arrays).

Reusability: MEDIUM. Division mode loading is E. coli-specific; subclass for platelets (or use empty division modes since platelets don't divide).

5.3.3 LocalEnvironment

Tracks external concentrations with media timeline support. Maps well to platelet agonist addition experiments.

Reusability: MEDIUM-HIGH.

5.4 Process Architecture

The base `Process` class (`wholecell/processes/process.py`) is fully generic:

```
class Process:
    def initialize(self, sim, sim_data): pass # Setup
    def calculateRequest(self): pass # Declare needs
    def evolveState(self): pass # Execute biology
    def isTimeStepShortEnough(self, dt, safety): return True
```

E. coli processes are **NOT reusable**—they serve only as examples.

5.5 Reusability Summary

Component	Generic?	Platelet Action
Simulation engine	YES	Use as-is, subclass
Process base class	YES	Use as-is
View system	YES	Use as-is
BulkMolecules	MOSTLY	Override ATP[c] hardcode
UniqueMolecules	MOSTLY	Subclass, custom division modes
LocalEnvironment	MOSTLY	Simplify exchange logic
Containers	YES	Use as-is
Listener base	YES	Use as-is
Cell division	NO	Not needed (platelets don't divide)
<i>E. coli</i> processes	NO	Write new platelet processes
<i>E. coli</i> reconstruction	NO	Write new platelet reconstruction

Table 5.1: Component reusability for platelet model.

5.6 Minimum sim_data Interface

For a minimal platelet simulation, `SimulationDataPlatelet` must provide:

```
class SimulationDataPlatelet:
    # Required by InternalState:
    submass_name_to_index = {'protein': 0, 'lipid': 1, 'small_molecule':
                             2}
    compartment_abbrev_to_index = {'c': 0, 'dg': 1, 'ag': 2, 'e': 3}

    # Required by BulkMolecules:
    constants.n_avogadro
    internal_state.bulk_molecules.bulk_data['id']
    internal_state.bulk_molecules.bulk_data['mass']
    molecule_groups.bulk_molecules_binomial_division # empty list OK

    # Required by UniqueMolecules:
    internal_state.unique_molecule.unique_molecule_definitions # can be
    {}
```

```
internal_state.unique_molecule.unique_molecule_masses

# Required by LocalEnvironment:
external_state.make_media
external_state.saved_media
```

Part III

The Platelet Model

Chapter 6

Biological Context

6.1 Key Differences from *E. coli*

Feature	<i>E. coli</i>	Platelet
Nucleus	Yes	No (anucleate)
Transcription	Active	None (minimal residual)
Replication	Continuous	None
Division	Binary fission	None
Lifespan	Indefinite (growth)	~7–10 days
Primary function	Growth & reproduction	Haemostasis & signalling
Time axis	Birth → division (60 min)	Resting → activated (seconds–minutes)

Table 6.1: Key biological differences between *E. coli* and platelets.

6.2 Platelet-Specific Biology to Model

6.2.1 Compartmentalisation

Platelets have highly organised internal compartments:

- **Dense granules (~3–8 per platelet):** Contain ADP, ATP, Ca^{2+} , serotonin
- **Alpha granules (~40–80 per platelet):** Contain fibrinogen, vWF, growth factors
- **Open canalicular system:** Membrane invaginations for secretion
- **Mitochondria:** ATP production, calcium buffering
- **Cytosol:** Signalling cascades, cytoskeleton

6.2.2 Receptors and Signalling

Major platelet receptors:

- **GPVI:** Collagen receptor (ITAM signalling)
- **PAR1/PAR4:** Thrombin receptors (GPCR)
- **P2Y1/P2Y12:** ADP receptors (GPCR)
- **$\alpha\text{IIb}\beta 3$:** Fibrinogen receptor (integrin)

All converge on calcium mobilisation as the central integrator.

6.2.3 Calcium Dynamics

Ca^{2+} signalling involves:

1. Receptor activation $\rightarrow \text{PLC}\gamma 2 \rightarrow \text{IP}_3$ production
2. $\text{IP}_3 \rightarrow \text{ER}$ calcium release (STIM1/Orai1)
3. Store-operated calcium entry (SOCE)
4. Mitochondrial Ca^{2+} uptake (MCU) and release (NCLX)
5. Plasma membrane Ca^{2+} ATPase (PMCA) efflux

6.2.4 Granule Exocytosis

Calcium-dependent secretion:

- SNARE machinery (VAMP-3/8, syntaxin-11, SNAP-23)
- Different Ca^{2+} thresholds for α - vs δ -granules
- Positive feedback via released ADP

6.2.5 Metabolism

Platelets rely on:

- Glycolysis (primary ATP source at rest)
- Oxidative phosphorylation (mitochondria)
- Metabolic switching upon activation (PDK/PDH)

Chapter 7

Literature Analysis

7.1 Key Data Sources

Based on analysis of ~ 90 papers across four Zotero collections:

7.1.1 Calcium Dynamics

High Priority:

- **Dolan & Diamond (2014):** 34-species ODE model of Ca^{2+} signalling—the most comprehensive implementable model
- **Kleppe (2018):** ODE model of NO/cGMP/cAMP inhibitory pathway (missing from other Ca^{2+} models)
- **Ghatge (2026):** MCU (mitochondrial calcium uniporter) quantification
- **Fernández (2021):** Ca^{2+} threshold for TMEM16F/anoctamin-6 activation

7.1.2 Receptor Signalling

High Priority:

- **Dunster (2015):** Data-driven ODE model of $\text{GPVI} \rightarrow \text{Syk} \rightarrow \text{PLC}\gamma 2$ pathway
- **Mazet (2020):** ODE model of PI cycle ($\text{PIP}_2 \rightarrow \text{IP}_3 + \text{DAG}$)

7.1.3 Granule Exocytosis

High Priority:

- **Fitch-Tewfik & Flaumenhaft (2013):** SNARE machinery and fusion pore dynamics
- **Lopez et al. (2015):** Ca^{2+} thresholds for α - vs δ -granule secretion

7.1.4 Metabolism

High Priority:

- **Thomas et al. (2014):** iAT-PLT-636 constraint-based metabolic reconstruction (636 reactions)
- **Flora et al. (2023):** PDK/PDH metabolic switch mechanism

7.1.5 Proteomics

Critical Resource:

- **Burkhart et al. (2012):** Quantitative platelet proteome—copy numbers for $\sim 4,000$ proteins

Chapter 8

Implementation Plan

8.1 Phased Approach

8.1.1 Phase 1: Framework Validation (2–3 weeks)

Goal: Confirm wcEcoli engine runs a minimal non-E. coli simulation.

1. Clone and run standard E. coli simulation end-to-end
2. Map the architecture (processes, states, data flow)
3. Document minimum `sim_data` interface requirements

Deliverable: Architecture documentation (complete—see Part II).

8.1.2 Phase 2: Minimal Platelet Stub (3–4 weeks)

Goal: Create a runnable “blank cell” in the wcEcoli framework.

1. Create `models/platelet/` namespace with:
 - `sim/simulation.py` — `PlateletSimulation` class
 - `sim/initial_conditions.py` — Set starting state
 - One toy process (e.g., `RestingDecay`)
 - One listener (e.g., `Mass`)
2. Create `reconstruction/platelet/` with:
 - `simulation_data.py` — `SimulationDataPlatelet`
 - Minimal molecule definitions
 - Compartment definitions (cytosol, granules, extracellular)
3. Verify simulation runs without E. coli dependencies

Deliverable: Runnable platelet stub (~500 lines new code).

8.1.3 Phase 3: Biological Processes (8–12 weeks)

Phase 3a: Calcium Dynamics (3–4 weeks)

Implement ODE-based calcium process:

- ER calcium stores (IP₃R, SERCA)
- Mitochondrial buffering (MCU, NCLX)
- Plasma membrane fluxes (PMCA, SOCE)
- Base on Dolan & Diamond 34-species model + Kleppe inhibitory pathway

Validation: Reproduce published Ca²⁺ transients for thrombin stimulation.

Phase 3b: Granule Release (2–3 weeks)

Implement granule exocytosis:

- Granules as unique molecules with state (loaded/fusing/released)
- Ca^{2+} -dependent fusion rate function
- Cargo release into extracellular compartment
- Different thresholds for α - vs δ -granules

Validation: Reproduce secretion kinetics from Lopez et al. (2015).

Phase 3c: Receptor Signalling (2–3 weeks)

Implement receptor $\rightarrow \text{Ca}^{2+}$ cascade:

- One receptor pathway initially (GPVI or thrombin/PAR)
- Connect to IP_3 production
- Trigger calcium dynamics from receptor occupancy

Validation: Dose-response curves for agonist $\rightarrow \text{Ca}^{2+}$ mobilisation.

Phase 3d: Metabolism (2–3 weeks)

Implement simplified metabolic process:

- ATP production (glycolysis + OXPHOS)
- ATP consumption by processes
- Metabolic switch upon activation

Consider adapting iAT-PLT-636 reconstruction (Thomas et al. 2014).

8.1.4 Phase 4: Integration and Validation (4+ weeks)

1. Cross-validate processes (Ca^{2+} drives granule release drives ADP feedback)
2. Compare to experimental time courses
3. Sensitivity analysis on key parameters
4. Document limitations and future directions

8.2 v0.1 Target Definition

A concrete, falsifiable deliverable:

Simulate thrombin-induced Ca^{2+} rise and dense granule release over 5 minutes. Plot secretion kinetics and compare to published data.

This requires:

- Working simulation engine with platelet namespace

- Calcium dynamics process (ODE)
- Granule release process
- Thrombin receptor signalling (minimal)
- Listeners for Ca^{2+} trace and granule counts

8.3 Proposed File Structure

```

models/platelet/
  sim/
    simulation.py          # PlateletSimulation(Simulation)
    initial_conditions.py  # Set initial state
  processes/
    calcium_dynamics.py   # Ca2+ store release, buffering
    granule_release.py    # Exocytosis
    receptor_signalling.py # GPVI, PAR, P2Y12 -> Ca2+
    metabolism.py         # ATP production/consumption
  listeners/
    mass.py
    activation_state.py
    granule_counts.py
    calcium_trace.py

reconstruction/platelet/
  simulation_data.py       # SimulationDataPlatelet
  knowledge_base_raw.py    # Literature values
  dataclasses/
    constants.py
    molecule_groups.py
  state/
    internal_state.py
    external_state.py

```

Chapter 9

Feasibility Assessment

9.1 Technical Feasibility

9.1.1 Framework Suitability

The wcEcoli framework is well-suited for adaptation:

- **Modular design:** Clear separation between engine and organism-specific code
- **Documented interfaces:** State and process APIs are well-defined
- **Proven at scale:** Handles 3,000+ molecules, 15+ processes
- **Existing tooling:** Analysis scripts, data I/O, logging infrastructure

9.1.2 Biological Compatibility

Key adaptations required:

- **No division:** Set `_divideCellFunction = None` (straightforward)
- **ODE processes:** Replace FBA with kinetic ODEs for signalling (requires new process implementations)
- **Time axis:** Seconds–minutes activation rather than hours growth (adjust `lengthSec`)

9.1.3 Risk Assessment

Risk	Likelihood	Impact	Mitigation
Framework too E. coli-specific	Low	High	Phase 2 stub validates genericity
Insufficient parameter data	Medium	Medium	Use Burkhardt proteomics + literature
ODE stiffness issues	Medium	Low	Use adaptive solvers (scipy)
Scope creep	High	Medium	Strict v0.1 target definition

Table 9.1: Risk assessment for platelet model development.

9.2 Scientific Feasibility

9.2.1 Data Availability

Platelet biology is well characterised:

- **Proteomics:** Burkhardt et al. provides copy numbers for ~4,000 proteins
- **Signalling kinetics:** Multiple ODE models available (Dolan, Dunster, Kleppe)

- **Metabolomics:** Metabolic reconstructions exist (Thomas et al.)
- **Validation data:** Extensive experimental literature on activation kinetics

9.2.2 Model Scope

A proof-of-concept model does not require:

- Complete coverage of all pathways (start with 1–2 receptors)
- Spatial resolution (well-mixed is adequate initially)
- Full metabolic network (simplified ATP module sufficient)

9.3 Resource Requirements

9.3.1 Computational

- **Development:** Standard laptop sufficient
- **Simulation:** Expected <10 minutes per 5-minute simulation
- **No GPU required** (unlike MC4D)

9.3.2 Time Estimate

Phase	Duration	Cumulative
Phase 1: Framework validation	2–3 weeks	3 weeks
Phase 2: Minimal stub	3–4 weeks	7 weeks
Phase 3a: Calcium dynamics	3–4 weeks	11 weeks
Phase 3b: Granule release	2–3 weeks	14 weeks
Phase 3c: Receptor signalling	2–3 weeks	17 weeks
Phase 3d: Metabolism	2–3 weeks	20 weeks
Phase 4: Integration	4+ weeks	24 weeks

Table 9.2: Estimated timeline for proof-of-concept model.

9.4 Conclusion

Building a whole-cell model of the human platelet using the wcEcoli framework is **technically feasible** and **scientifically justified**:

1. The wcEcoli framework provides a mature, well-tested foundation with explicit support for reuse.
2. Platelet biology is well characterised with sufficient data for parameterisation.
3. A proof-of-concept model (Ca^{2+} dynamics + granule release) is achievable in ~ 6 months.
4. The model would fill a gap in the field—no integrative WCM of platelets currently exists.

The key success factors are:

- **Ruthless scope control:** Start with minimal biology, validate, then extend
- **ODE-first approach:** Use kinetic models for signalling rather than FBA
- **Clear validation targets:** Define falsifiable predictions from the outset

Part IV

Appendices

Appendix A

Key Literature References

A.1 Whole-Cell Modelling

1. Karr JR et al. (2012) A whole-cell computational model predicts phenotype from genotype. *Cell* 150(2):389–401.
2. Macklin DN et al. (2020) Simultaneous cross-evaluation of heterogeneous E. coli datasets. *Science* 369:eaav3751.
3. Thornburg ZR et al. (2026) Bringing the genetically minimal cell to life on a computer in 4D. *Cell*.

A.2 Platelet Calcium Signalling

1. Dolan AT, Diamond SL (2014) Systems modeling of Ca²⁺ homeostasis and mobilization in platelets. *PLoS ONE*.
2. Kleppe LS (2018) Mathematical modelling of NO/cGMP/cAMP signalling in platelets.
3. Ghatge M et al. (2026) The mitochondrial calcium uniporter regulates calcium dynamics.

A.3 Platelet Receptor Signalling

1. Dunster JL et al. (2015) Regulation of early steps of GPVI signal transduction by phosphatases. *PLoS Comput Biol*.
2. Mazet F et al. (2020) A model of the PI cycle reveals regulating roles of lipid-binding proteins.

A.4 Platelet Metabolism

1. Thomas SG et al. (2014) Network reconstruction of platelet metabolism. *Blood*.
2. Flora GD et al. (2023) Mitochondrial PDK2/4 contribute to platelet function.

A.5 Platelet Proteomics

1. Burkhardt JM et al. (2012) The first comprehensive and quantitative analysis of human platelet protein composition. *Blood* 120(15):e73–e82.

Appendix B

Glossary

FBA Flux Balance Analysis — constraint-based metabolic modelling

RDME Reaction-Diffusion Master Equation — spatially-resolved stochastic simulation

CME Chemical Master Equation — well-mixed stochastic simulation

ODE Ordinary Differential Equations — deterministic kinetic modelling

ParCa Parameter Calculator — wcEcoli’s data fitting pipeline

GPVI Glycoprotein VI — platelet collagen receptor

PAR Protease-Activated Receptor — thrombin receptor

SOCE Store-Operated Calcium Entry

MCU Mitochondrial Calcium Uniporter

SNARE Soluble NSF Attachment Protein Receptor — membrane fusion machinery